Practically Groovy: Go server-side up, with Groovy

On-the-fly server-side programming with Groovlets and GSPs

Skill Level: Intermediate

Andrew Glover (andrew@thirstyhead.com)
Co-Founder
ThirstyHead.com

Scott Davis (scott@thirstyhead.com)
Founder
ThirstyHead.com

15 Mar 2005

The Groovlet and GroovyServer Pages (GSP) frameworks are built on the shoulders of the Java™ Servlet API. Unlike Struts and JSF, however, Groovy's server-side implementation isn't meant for all occasions. Rather, it's a simplified alternative for developing server-side applications quickly and easily. Follow along with Groovy advocate Andrew Glover as he introduces these frameworks and demonstrates their use.

The Java platform arguably has made a name for itself as the platform of choice for server-side application development. Servlets have been a strong foothold for server-side Java technology, so much so that myriad frameworks have been built around the Servlets API, including Struts, JavaServer Faces (JSF), and Tapestry, to name a few. As you've probably guessed, Groovy has also built a framework on the shoulders of the Servlets API; however, the aim of this framework is simplicity.

The Groovlet and GroovyServer Pages (GSP) frameworks aim to provide an elegant yet simple platform for building Web applications of minimal complexity. Just as GroovySql shouldn't be your only choice for database development, the Groovlet framework *is not* a replacement for more feature-rich frameworks like Struts. Groovlets are simply an alternative for developers seeking an easy configuration and quick means to producing working code.

For example, a short time ago, I needed to provide -- *quickly* -- a stub application for testing the client side of an `xml-rpc`-like API. It was obvious I could rapidly stub out the required functionality with a servlet, but I would have never considered delving into Struts -- not even for a second. I considered writing the servlet and its associated logic using the base normal-Java Servlet API; but because I needed the functionality ASAP, I chose to knock it out using Groovlets.

As you'll see shortly, the choice was obvious.

Before I get too far into programming with Groovlets, I'd like to quickly review a Groovy feature that will come up in the example code. I first wrote about the `def` keyword in the original Feeling Groovy article, a few months back.

About this series

The key to incorporating any tool into your development practice is knowing when to use it and when to leave it in the box. Scripting languages can be an extremely powerful addition to your tool kit, but only when applied properly to appropriate scenarios. To that end, *Practically Groovy* is a series of articles dedicated to exploring the practical uses of Groovy, and teaching you when and how to apply them successfully.

Defining functions in scripts

In normal Java programming, methods must exist within a class object. In fact, all behavior must be defined within the context of a class. In Groovy, however, behavior can be defined within *functions*, which can be defined outside a class definition.

These functions can be referenced directly by name and can be defined in Groovy scripts, where they can seriously facilitate reuse. Groovy functions require the `def` keyword, and you can think of them as globally static methods available within a script's scope. Because Groovy is a dynamically typed language, `def`s do not require any type declarations for parameters, nor do `def`s require a `return` statement.

For example, in Listing 1, I define a simple function that prints out the contents of a collection, be it a `list` or a `map`. I then proceed to define a `list`, populate it, and call my newly defined `def`. Next, I create a `map` and do the same thing for that collection.

Listing 1. Now that's def!

```
def logCollection(coll){
  coll.eachWithIndex{ element, i ->
    println "Item ${i}: ${element}"
  }
```

```
}
def lst = [12, 3, "Andy", 'c']
logCollection(lst)
def mp = [name:"Groovy", date:new Date()]
logCollection(mp)
```

`defs` do not require a `return` statement, so if the last line produces some value, that value is returned by the `def`. For example, in Listing 2, the code defines a `def` that returns the class name of the variable passed in. I can write it with or without the `return` statement, and my results will be the same.

Listing 2. Return statements are optional in defs

```
def getJavaType(val){
  val.class.canonicalName
}
def tst = "Test"
println getJavaType(tst)
```

The `def` keyword can be extremely handy when it comes to writing simple scripts. As you'll soon see, it can also be useful when developing Groovlets.

Groovlets and GSPs

The prerequisites for working with Groovlets and GSPs are quite simple: You need a servlet container, and the latest and greatest version of Groovy. The beauty of these frameworks is that they map all URLs of a chosen pattern to a specific servlet via a web.xml file. Thus, the first step to setting up a Groovlets and GSP implementation is to define a Web application context and update its associated web.xml file. The file will include the specific servlet class definitions and their corresponding URL pattern.

I'll be using Apache Jakarta Tomcat, and I've created a context called *groove*. The directory layout is shown in Listing 3:

Listing 3. Directory listing of the groove context

```
./groove:
drwxrwxrwx+   3 aglover  users        0 Jan 19 12:14 WEB-INF
./WEB-INF:
-rwxrwxrwx+   1 aglover  users      906 Jan 16 14:37 web.xml
drwxrwxrwx+   2 aglover  users        0 Jan 19 17:12 lib
./WEB-INF/lib:
-rwxrwxrwx+   1 aglover  users   832173 Jan 16 14:28 groovy-all-1.5.7.jar
```

In the WEB-INF directory, I need to have a web.xml file with at least the elements shown in Listing 4:

Listing 4. A fully configured web.xml file

```
<web-app xmlns="http://java.sun.com/xml/ns/javaee"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
        version="2.5">
        <display-name>Groove</display-name>
    <servlet>
      <servlet-name>GroovyServlet</servlet-name>
      <servlet-class>groovy.servlet.GroovyServlet</servlet-class>
    </servlet>
    <servlet>
        <servlet-name>GroovyTemplate</servlet-name>
        <servlet-class>groovy.servlet.TemplateServlet</servlet-class>
        </servlet>
    <servlet-mapping>
        <servlet-name>GroovyServlet</servlet-name>
        <url-pattern>*.groovy</url-pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>GroovyTemplate</servlet-name>
        <url-pattern>*.gsp</url-pattern>
    </servlet-mapping>
</web-app>
```

The definitions in the above web.xml file state that any request ending with `.groovy` (for example, `http://localhost:8080/groove/hello.groovy`) will be sent to the class `groovy.servlet.GroovyServlet`, while any request ending with `.gsp` will go to the class `groovy.servlet.TemplateServlet`.

My next step is to place the Groovy jar (in my case, groovy-all-1.5.7.jar) in the lib directory.

Yep, it's that easy — I'm ready to go.

The Groovlet, please

Writing Groovlets is indubitably simple, since Groovy poses few requirements in terms of class hierarchy extensions. With Groovlets, there is no need to extend from `javax.servlet.http.HttpServlet`, `javax.servlet.GenericServlet`, or some slick `GroovyServlet` class. In fact, creating a Groovlet is as simple as creating a Groovy script. You don't even have to create a class. In Listing 5, I've written a simple Groovlet that does two things: It prints some HTML, then it provides some information about the container it's running in.

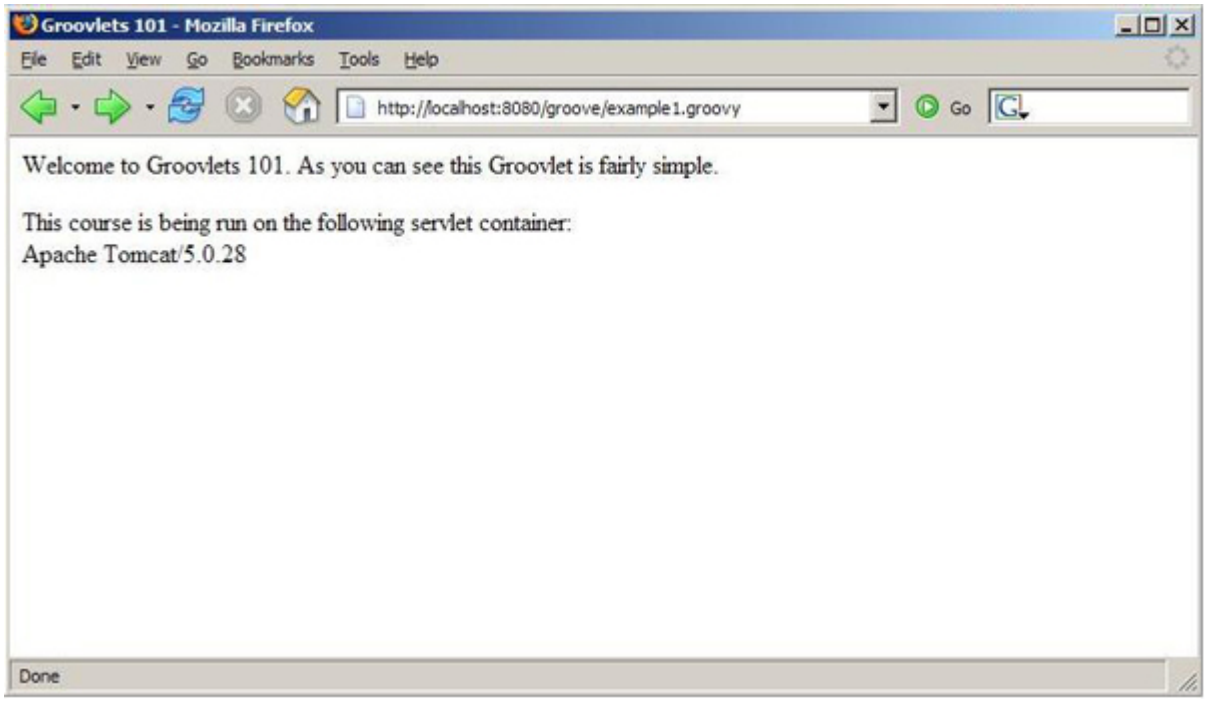
Listing 5. Getting started with Groovlets

```
println """
<html><head>
<title>Groovlets 101</title>
</head>
```

```
<body>
<p>
Welcome to Groovlets 101. As you can see
this Groovlet is fairly simple.
</p>
<p>
This course is being run on the following servlet container: </br>
${application.getServerInfo()}
</p>
</body>
</html>
"""
```

If you viewed this Groovlet in your browser, it would look something like what you see in Figure 1.

Figure 1. Output from the simple Groovlet



Looking closely at the Groovlet in Listing 5 should take you back to the time when you first started writing Groovy scripts. First, there is no `main` method or class definition, just some simple code. What's more, the Groovlet framework implicitly provides instance variables, such as `ServletRequest`, `ServletResponse`, `ServletContext`, and `HttpSession`. See how I was able to reference the instance of `ServletContext` via the `application` variable? If I wanted to grab the instance of `HttpSession`, I'd use the `session` variable name. Similarly, I can use `request` and `response` for `ServletRequest` and `ServletResponse`, respectively.

A diagnostic Groovlet

Not only is writing a Groovlet as simple as creating a Groovy script but you can also define functions with the `def` keyword and call them directly within the Groovlet. To demonstrate, I'll create a nontrivial Groovlet that performs some diagnostic checks for a Web application.

Imagine you've written a Web application that has been purchased by various customers around the world. You have a large customer base and have been releasing this application for some time now. Learning from past support issues, you've noticed that you receive many frantic customer calls related to a problem stemming from incorrect JVM versions and incorrect object-relational mapping (ORM).

You're busy, so you ask me to come up with a solution. Using Groovlets, I *quickly* can create a simple diagnostic script that verifies the VM version and attempts to create a Hibernate session (see Resources). I start by creating two functions and calling them when the script is hit via a browser. The diagnostic Groovlet is defined in Listing 6:

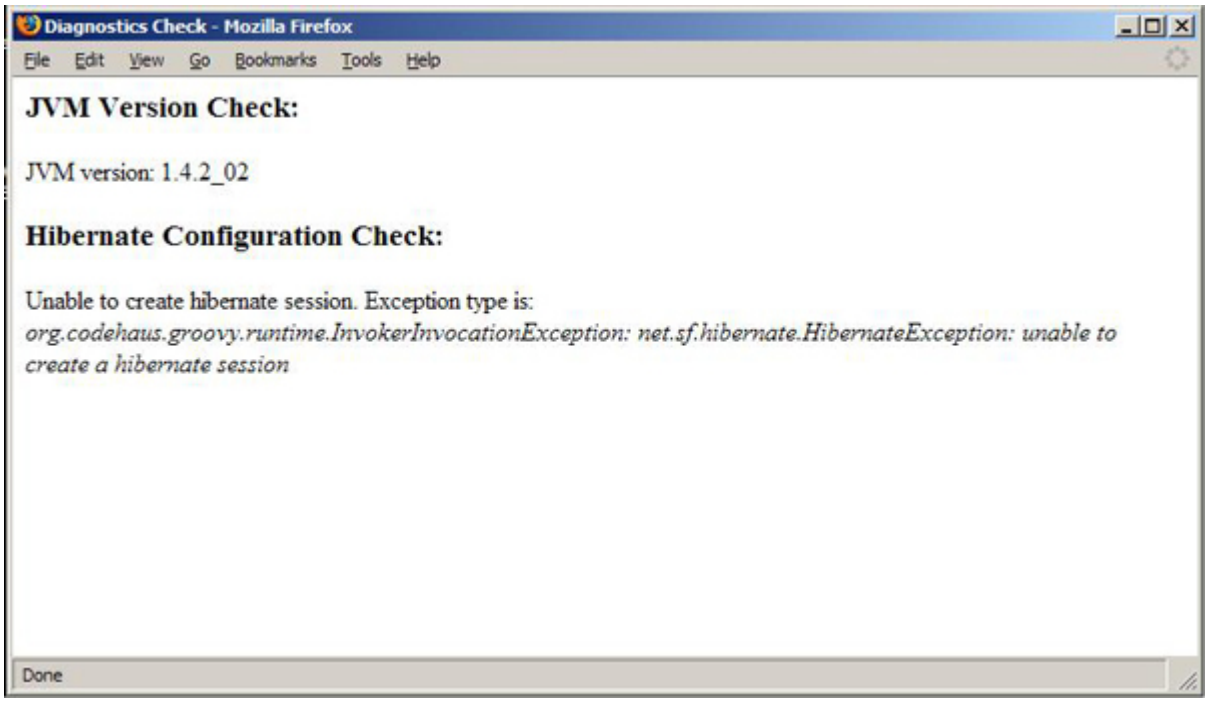
Listing 6. A diagnostic Groovlet

```
/**
 * Tests VM version from environment- note, even 1.5 will
 * cause an assertion error.
 */
def testVMVersion(){
  println "<h3>JVM Version Check: </h3>"
  def vers = System.getProperty("java.version")
  println "<p>JVM version: ${vers} </p>"
}

/**
 * Attempts to create an instance of a hibernate session. If this
 * works we have a connection to a database; additionally, we
 * have a properly configured hibernate instance.
 */
def testHibernate(){
  println "<h3>Hibernate Configuration Check: </h3>"
  try{
    def sessFactory = DefaultHibernateSessionFactory.getInstance()
    def session = sessFactory.getHibernateSession()
    println "<p>Hibernate configuration check was successful</p>"
  }catch(Throwable tr){
    println """
    <p>Unable to create hibernate session. Exception type is: <br/>
    <i>${tr.toString()} </i><br/>
    </p>
    """
  }
}

println """
<html>
  <head>
    <title>Diagnostics Check</title>
  </head>
  <body>
"""
testVMVersion()
```

```
testHibernate()
println """
  </body>
</html>
"""
```

The Groovlet's verification logic is fairly simple, but it will do the trick. You'll simply bundle the diagnostic script with your Web application, and when your customer support desk receives a call, they'll point the customer to the `Diagnostics.groovy` script in their browser and ask them to report their findings. The results could look something like what you see in Figure 2.

Figure 2. Output from the diagnostic Groovlet



What about those GSPs?

So far, I've focused on writing Groovlets. As you'll see, however, Groovy's GSP pages easily complement the Groovlets framework, much like JSPs complement the Servlet API.

On the surface, GSPs look just like JSPs, but actually, they couldn't be more different because the GSP framework is really a template engine. If you're unfamiliar with template engines, you might want to quickly review last month's article.

While GSPs and JSPs are fundamentally different technologies, they are similar in that GSPs are excellent candidates for embodying the view of a Web application. As you might recall from last month, a view-facilitating technology lets you separate the

concerns of your application's business logic and its corresponding view. If you take a quick look back at the diagnostic Groovlet in Listing 6, you might see where it could stand to be improved with GSP code.

Yep, that Groovlet is kind of ugly, isn't it? The problem is that it mixes application logic and a load of `println`s to output the HTML. Fortunately, I can resolve the situation by creating a simple GSP to complement the Groovlet.

An example GSP

Creating GSPs is as easy as creating Groovlets. The key to GSP development is realizing that a GSP is essentially a template and, therefore, may be best served by limited logic. I'll create a simple GSP in Listing 7 to get us started:

Listing 7. A simple GSP

```
<html>
  <head><title>A Simple GSP</title></head>
  <body>
    <b><% println "hello gsp" %></b>
    <p>
    <% def wrd = "Groovy"
       wrd.each{ letter ->
     %>
     <h1> <%= letter %> <br/>
     <%} %>
    </p>
  </body>
</html>
```

Looking at the above GSP should seriously remind you of standard Groovy template development. The syntax is JSP-ish in its use of <%s, but, like the Groovlet framework, it lets you access common servlet objects, such as `ServletRequest`, `ServletResponse`, `ServletContext`, as well as `HttpSession` objects.

Refactor me this ...

You can learn a lot from refactoring older code as your experience with a programming language or platform grows. I'd like to revisit the simple reporting application from January's column, when you were just learning learning about GroovySql.

As you'll recall, I built a quick-and-dirty reporting application that could have multiple uses within an organization. As it turns out, the application has since become quite popular for studying activity on the company database. Now, nontechnical personnel want to have access to this stupendous report, but they don't want the overhead of having to install Groovy on their machines to run it.

I kind of predicted this would happen, and the solution seems *practically* obvious: I'll Web-enable the reporting application. Lucky for me, Groovlets and GSPs will make the refactoring a snap.

Refactoring the reporting application

First, I'll tackle the guts of the simple application from Listing 12 of the GroovySql article. Refactoring this is easy: I simply replace all the `println`s with logic that places an instance variable in the `HttpRequest` object using the `setAttribute()` method.

My next step is to forward the `request`, using a `RequestDispatcher`, to a GSP that will handle the view component of the reporting application. The new report Groovlet is defined in Listing 8:

Listing 8. The refactored database reporting application

```
import groovy.sql.Sql
/**
 * forwards to passed in page
 */
def forward(page, req, res){
  def dis = req.getRequestDispatcher(page);
  dis.forward(req, res);
}
def sql = Sql.newInstance("jdbc:mysql://localhost:3306/words", "words",
        "words", "com.mysql.jdbc.Driver")

def uptime = ""
def questions = ""
sql.eachRow("show status"){ status ->
  if(status.variable_name == "Uptime"){
     uptime =  status[1]
     request.setAttribute("uptime", uptime)
  }else if (status.variable_name == "Questions"){
     questions =  status[1]
     request.setAttribute("questions", questions)
  }
}

request.setAttribute("qpm", Integer.valueOf(questions) /
Integer.valueOf(uptime) )

int insertnum = 0
int selectnum = 0
int updatenum = 0
sql.eachRow("show status like 'Com_%'"){ status ->
  if(status.variable_name == "Com_insert"){
     insertnum =  Integer.valueOf(status[1])
  }else if (status.variable_name == "Com_select"){
     selectnum =  Integer.valueOf(status[1])
  }else if (status.variable_name == "Com_update"){
     updatenum =  Integer.valueOf(status[1])
 }
}

request.setAttribute("qinsert", 100 * (insertnum / Integer.valueOf(uptime)))
request.setAttribute("qselect", 100 * (selectnum / Integer.valueOf(uptime)))
request.setAttribute("qupdate", 100 * (updatenum / Integer.valueOf(uptime)))
forward("mysqlreport.gsp", request, response)
```

The code in Listing 8 should be quite familiar. I've simply replaced all the `println`s from the previous application and added a `forward` function to handle the view portion of the report.

Adding the view component

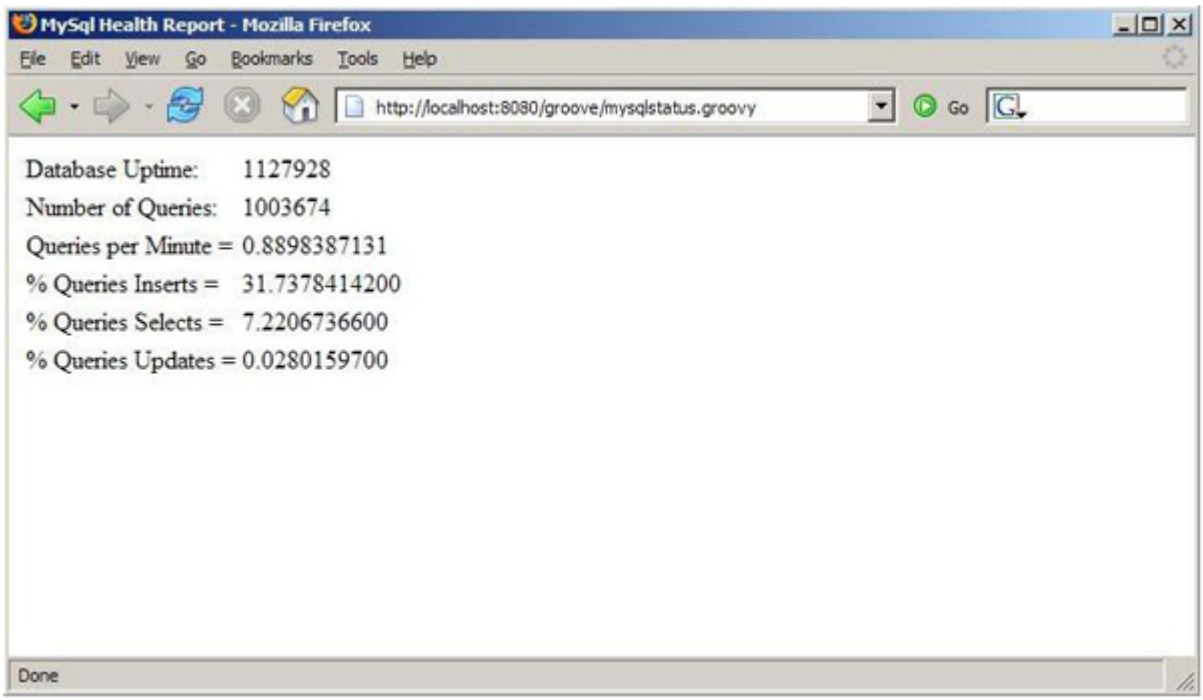
My next step is to create the GSP to handle the reporting application's view. Because I'm an engineer and not an artist, my view is rather simple -- a tad of HTML with a table, as shown in Listing 9:

Listing 9. The view component of the report

```
<html>
  <head>
    <title>MySql Health Report</title>
  </head>
  <body>
    <table>
      <tr>
        <td>Database Uptime:</td><td><%=
        "${request.getAttribute("uptime")}" %></td>
      </tr>
      <tr>
        <td>Number of Queries:</td><td><%=
        "${request.getAttribute("questions")}" %></td>
      </tr>
      <tr>
        <td>Queries per Minute =</td><td><%=
        "${request.getAttribute("qpm")}" %></td>
      </tr>
      <tr>
        <td>% Queries Inserts =</td><td><%=
        "${request.getAttribute("qinsert")}" %></td>
      </tr>
      <tr>
        <td>% Queries Selects =</td><td><%=
        "${request.getAttribute("qselect")}" %></td>
      </tr>
      <tr>
        <td>% Queries Updates =</td><td><%=
        "${request.getAttribute("qupdate")}" %></td>
      </tr>
    </table>
  </body>
</html>
```

Running the new report should result in the output found in Listing 3. Of course, numbers will vary.

Figure 3. Output from the refactored reporting application



Conclusion

As you can see, Groovlets and GSPs are the obvious choice for server-side development when the required functionality is fairly simple and needs to have been built yesterday. Both frameworks are extremely flexible and the code-to-view turnaround time is virtually unbeatable.

Let me stress, however, that Groovlets are not a replacement for Struts. The GSP framework isn't competing *directly* with Velocity. GroovySql isn't a replacement for Hibernate. And Groovy isn't a replacement for the Java language.

In all cases, these technologies are complementary, and in most cases, Groovy is the simpler alternative for on-the-fly development. Just like GroovySql is an alternative to using JDBC directly, Groovlets and GSPs are practical alternatives to using the Servlet API directly.

Next month, I'll delve deeper into the wonderful world of GroovyMarkup.

Downloads

Description	Name	Size	Download method
Sample code	j-pg03155.zip	3MB	HTTP

Information about download methods

About the authors

Andrew Glover
Andrew Glover is a developer, author, speaker, and entrepreneur. He is the founder of the easyb Behavior-Driven Development (BDD) framework and is the co-author of three books: Continuous Integration, Groovy in Action, and Java Testing Patterns. He teaches a wide variety of Groovy-, Grails-, and testing-related classes at ThirstyHead.com. You can keep up with Andy at thediscoblog.com where he routinely blogs about software development.

Scott Davis
Scott Davis is an internationally recognized author, speaker, and software developer. He is the founder of ThirstyHead.com, a Groovy and Grails training company. His books include Groovy Recipes: Greasing the Wheels of Java, GIS for Web Developers: Adding Where to Your Application, The Google Maps API, and JBoss At Work. He writes two ongoing article series for IBM developerWorks: Mastering Grails and Practically Groovy.